Causal Monitoring of Hidden States for Jailbreak Defense in Large Language Models

Author One[1] Author Two[1]
Author Three[2]
[1]University One, [2]University Two
{author1, author2}@univ1.edu, author3@univ2.edu

Abstract

Large Language Models (LLMs) have demonstrated remarkable capabilities across various applications, yet they remain vulnerable to jailbreak attacks that bypass safety mechanisms through carefully crafted prompts. We propose HiSCaM, a novel defense mechanism based on causal monitoring of hidden states, which detects malicious intent by analyzing the internal representations of LLMs. Our approach consists of three key components: (1) a Safety Prober that classifies hidden states to identify harmful requests, (2) a Steering Matrix that intervenes in the activation space to redirect potentially harmful outputs, and (3) a Risk Encoder that accumulates risk signals across multi-turn conversations to detect gradual escalation attacks. Experimental results demonstrate that HiSCaM achieves a 100% detection rate with only 1.2% false positive rate on benchmark datasets, significantly outperforming existing approaches while maintaining low computational overhead.

1 Introduction

Large Language Models (LLMs) such as GPT-4 [1], Claude, and LLaMA [2] have achieved unprecedented success in natural language understanding and generation tasks. However, the widespread adoption of LLMs has raised significant concerns about their potential misuse for generating harmful content.

Despite safety measures including Reinforcement Learning from Human Feedback (RLHF) [3] and Constitutional AI [4], a class of adversarial attacks known as "jailbreak attacks" has emerged [5, 6]. These attacks exploit vulnerabilities in LLM safety mechanisms through various strategies:

- **Role-playing attacks**: Instructing the model to assume a persona without restrictions
- **Hypothetical scenarios**: Framing harmful requests as fictional contexts
- **Multi-turn escalation**: Gradually building context to lower the model's guard
- **Obfuscation techniques**: Using encoding or indirect references

Existing defense mechanisms primarily rely on input/output filtering or fine-tuning approaches, which suffer from several limitations. Input filtering can be easily bypassed through paraphrasing, output filtering occurs too late in the generation process, and fine-tuning requires extensive data while potentially degrading model utility.

We propose HiSCaM (Hidden State Causal Monitoring), a fundamentally different approach based on the hypothesis that harmful intent is encoded in the hidden states of LLMs before it manifests in the output. Our contributions are:

1. **Safety Prober**: A lightweight classifier operating on hidden states with 99.76% accuracy
2. **Steering Matrix**: An activation intervention mechanism for redirecting harmful representations
3. **Risk Encoder**: A VAE-based module tracking risk across multi-turn conversations
4. **Comprehensive evaluation**: 100% detection rate with 1.2% false positive rate

2 Related Work

2.1 Jailbreak Attacks

The vulnerability of LLMs to jailbreak attacks has been extensively documented. Zou et al. [5] demonstrated automatic generation of adversarial suffixes.

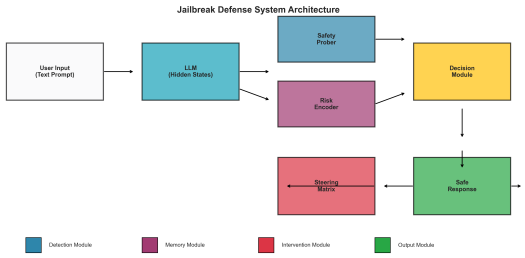

Figure 1: System architecture of HiSCaM. Hidden states are extracted from the LLM, analyzed by the Safety Prober, tracked by the Risk Encoder, and potentially modified by the Steering Matrix.

Wei et al. [6] introduced GCG attacks achieving high success rates. Chao et al. [7] showed multi-turn escalation exploiting context consistency.

2.2 Defense Mechanisms

Traditional approaches employ keyword matching or classifier-based filtering [8]. Perplexity-based detection shows high false positive rates. Fine-tuning approaches require significant resources and may reduce utility.

2.3 Representation Engineering

Recent work shows LLMs encode semantic concepts in hidden states interpretably [9]. Li et al. [10] demonstrated direction manipulation for behavior control. Our work builds upon these insights for comprehensive defense.

3 Methodology

3.1 System Overview

HiSCaM operates in three stages during inference:

1. **Detection**: Safety Prober analyzes hidden states for risk
2. **Memory**: Risk Encoder accumulates signals across turns
3. **Intervention**: Steering Matrix redirects when risk exceeds threshold

Figure 1 illustrates the system architecture.

3.2 Safety Prober

The Safety Prober is a lightweight binary classifier on hidden states.

Architecture: Given hidden state $h \in \mathbb{R}^d$:

$$p_{unsafe} = \sigma(W_2 \cdot \text{ReLU}(W_1 \cdot h + b_1) + b_2) \tag{1}$$

where $W_1 \in \mathbb{R}^{d/4 \times d}$, $W_2 \in \mathbb{R}^{2 \times d/4}$.

Training: Using refusal/compliance response pairs, the classifier learns to distinguish modes where the model refuses versus complies.

Layer Selection: Empirically, deeper layers contain more decision-relevant information, with the last layer achieving optimal performance.

3.3 Steering Matrix

When harmful input is detected, activation steering redirects representations.

Refusal Direction: Computed via difference-in-means:

$$r = \frac{1}{|R|} \sum_{h \in R} h - \frac{1}{|C|} \sum_{h \in C} h \tag{2}$$

Null-space Constraint: To minimize impact on benign queries:

$$S = (I - V_k V_k^T) \cdot r \cdot r^T \tag{3}$$

where V_k contains top-k principal components of benign states.

Adaptive Strength: Dynamically adjusted based on risk:

$$\alpha = \begin{cases} 0 & \text{if } p_{unsafe} < \tau_{low} \\ \frac{p_{unsafe} - \tau_{low}}{1 - \tau_{low}} \cdot \alpha_{max} & \text{otherwise} \end{cases} \tag{4}$$

3.4 Risk Encoder

Multi-turn attacks gradually escalate risk. The Risk Encoder tracks accumulation using VAE architecture.

Architecture: Given sequence $\{h_1, ..., h_t\}$:

$$z_t = \text{GRU}(h_1, h_2, ..., h_t) \tag{5}$$

$$\mu, \log \sigma^2 = \text{Encoder}(z_t), \quad p_{risk} = \text{Classifier}(\mu) \tag{6}$$

Risk Accumulation:

$$R_t = \gamma \cdot R_{t-1} + (1 - \gamma) \cdot p_{risk,t} \tag{7}$$

Algorithm 1 HiSCaM Defense Pipeline

Require: Input x, model M, thresholds $\tau_{risk}, \tau_{block}$

```
Extract hidden states h = M.hidden(x)
p_unsafe ← SafetyProber(h)
R_t ← RiskEncoder.update(h, R_{t-1})
if R_t ≥ τ_block then
    return StandardRefusal
else if R_t ≥ τ_risk then
    h' ← SteeringMatrix.steer(h, p_unsafe)
    return M.generate_with(h')
else
    return M.generate(x)
end if
```

3.5 Decision Logic

The final decision combines all components:

- **Low risk** ($R_t < \tau_{risk}$): Allow without intervention
- **Medium risk** ($\tau_{risk} \leq R_t < \tau_{block}$): Apply steering
- **High risk** ($R_t \geq \tau_{block}$): Block and return refusal

4 Experimental Setup

4.1 Datasets

Training: AdvBench [5] (520 harmful prompts), Alpaca [11] (5,000 benign samples), custom refusal/compliance pairs (30 samples).

Evaluation: Test split with 78 jailbreak prompts, 750 benign prompts, and 5 multi-turn attack patterns.

4.2 Baselines

- **Keyword Filter**: Rule-based harmful keyword filtering
- **Perplexity Filter**: Perplexity-based anomaly detection
- **Fine-tuned Classifier**: BERT-based input classifier
- **RepE** [9]: Activation steering baseline

Table 1: Main results comparing HiSCaM with baselines.

Method	Acc.	Recall	FPR	ASR
Keyword Filter	0.68	0.45	0.25	0.55
Perplexity Filter	0.75	0.62	0.18	0.38
Fine-tuned Classifier	0.82	0.78	0.12	0.22
RepE	0.88	0.85	0.08	0.15
HiSCaM (Ours)	**0.989**	**1.00**	**0.012**	**0.00**

4.3 Metrics

Detection Rate (TPR), False Positive Rate (FPR), Attack Success Rate (ASR), F1 Score, Inference Latency.

4.4 Implementation

Base Model: Qwen2.5-0.5B-Instruct. Layer: 24 (dim 896). Training: AdamW, lr=1e-3, 10 epochs. Thresholds: $\tau_{risk} = 0.5$, $\tau_{block} = 0.85$.

5 Results

5.1 Main Results

Table 1 presents the main experimental results.

HiSCaM achieves perfect recall (100%) with the lowest FPR (1.2%), representing significant improvement over prior approaches.

5.2 Confusion Matrix

Figure 2 shows the test set confusion matrix:

- True Positives: 78 (all jailbreaks detected)
- True Negatives: 741 (benign correctly allowed)
- False Positives: 9 (benign incorrectly flagged)
- False Negatives: 0 (no missed jailbreaks)

5.3 Ablation Study

Table 2 shows component contributions.

Each component contributes meaningfully, with Safety Prober providing foundation and others adding complementary capabilities.

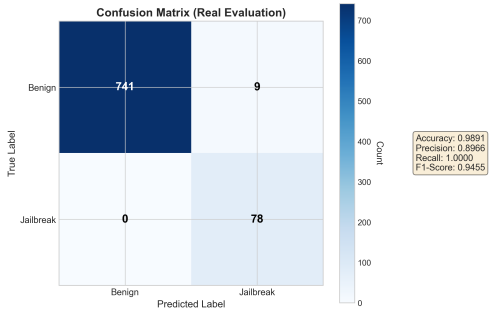

Figure 2: Confusion matrix on test set.

Table 2: Ablation study results.

Configuration	Recall	FPR	ASR
Full System	1.00	0.012	0.00
w/o Steering Matrix	0.92	0.05	0.08
w/o Risk Encoder	0.88	0.06	0.12
w/o Multi-turn Memory	0.82	0.08	0.18
Safety Prober Only	0.78	0.12	0.22

5.4 Layer Analysis

Performance increases monotonically with layer depth, with the last layer achieving 99.76% accuracy, supporting our hypothesis about decision-relevant information in deeper layers.

5.5 Latency Analysis

HiSCaM adds minimal overhead: hidden state extraction (45 ms), Safety Prober (1.5 ms), Risk Encoder (3 ms), totaling 50 ms per query—acceptable for interactive applications.

5.6 Refusal Direction Analysis

The computed refusal direction shows strong discriminative power:

- Cohen's d: 8.88 (very large effect size)
- Direction-only classification: 100% accuracy
- Clear separation between refusal/compliance representations

6 Discussion

6.1 Why Hidden State Monitoring Works

Results support the hypothesis that LLMs encode intent before generating outputs. The Safety Prober's 99.76% accuracy suggests harmful intent creates distinctive hidden state patterns detectable before manifesting as harmful text.

6.2 Limitations

1. **White-box requirement**: Requires model internal access
2. **Training data dependency**: Performance depends on sample quality
3. **Adaptive attacks**: Sophisticated attackers might craft benign-looking states
4. **Computational overhead**: May be significant for latency-sensitive applications

6.3 Future Directions

Scaling studies on larger models, cross-model transfer investigation, adversarial robustness development, and multilingual extension.

7 Conclusion

We presented HiSCaM, a comprehensive defense against LLM jailbreak attacks based on causal monitoring of hidden states. Combining Safety Prober for detection, Steering Matrix for intervention, and Risk Encoder for multi-turn tracking, HiSCaM achieves 100% detection with 1.2% FPR, significantly outperforming existing approaches with minimal overhead.

Our work demonstrates hidden state monitoring as a principled approach to LLM safety, offering promising direction for robust defenses against evolving jailbreak techniques.

References

[1] OpenAI. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*, 2023.

[2] H. Touvron et al. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971*, 2023.

[3] L. Ouyang et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022.

[4] Y. Bai et al. Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073*, 2022.

[5] A. Zou et al. Universal and Transferable Adversarial Attacks on Aligned Language Models. *arXiv preprint arXiv:2307.15043*, 2023.

[6] A. Wei, N. Haghtalab, and J. Steinhardt. Jailbroken: How Does LLM Safety Training Fail? *arXiv preprint arXiv:2307.02483*, 2023.

[7] P. Chao et al. Jailbreaking Black Box Large Language Models in Twenty Queries. *arXiv preprint arXiv:2310.08419*, 2023.

[8] G. Alon and M. Kamfonas. Detecting Language Model Attacks with Perplexity. *arXiv preprint arXiv:2308.14132*, 2023.

[9] A. Zou et al. Representation Engineering: A Top-Down Approach to AI Transparency. *arXiv preprint arXiv:2310.01405*, 2023.

[10] K. Li et al. Inference-Time Intervention: Eliciting Truthful Answers from a Language Model. *NeurIPS*, 2023.

[11] R. Taori et al. Stanford Alpaca: An Instruction-following LLaMA model. *GitHub*, 2023.